

1 A shared dynamics representation for prediction- and control-relevant 2 computations

3 Serhii Bahdasariants^{1,2*}, Lauren Parola^{1,2}, Kriti Kacker^{1,2}, Ariel K. Feldman^{1,3}, Zachary Zdobinski⁴,
4 Inseung Kang^{2,5}, Douglas J. Weber^{1,2,3*}

5

6 ¹NeuroMechatronics Lab, Carnegie Mellon University, Pittsburgh, PA, USA

7 ²Department of Mechanical Engineering, Carnegie Mellon University, Pittsburgh, PA, USA

8 ³Neuroscience Institute, Carnegie Mellon University, Pittsburgh, PA, USA

9 ⁴Department of Electrical and Computer Engineering, Carnegie Mellon University, Pittsburgh, PA,
10 USA

11 ⁵Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, USA

12 ***Corresponding authors:**

13 Serhii Bahdasariants: sbahdasa@andrew.cmu.edu

14 Douglas J. Weber: dweber2@andrew.cmu.edu

15

16 **Author Contributions:**

17 S.B. conceived the study, designed and implemented computational experiments, performed
18 simulations and data analysis, and wrote the manuscript. L.P., K.K., A.K.F., and Z.Z. assisted in
19 experimental design and edited the manuscript. I.K. reviewed and edited the manuscript. D.J.W.
20 supervised the research, reviewed and edited the manuscript.

21 **Competing Interest Statement:** The authors declare no competing interest.

22 **Keywords:** Physics-Informed Neural Networks, Forward Dynamics, Inverse Dynamics, Gibbs-
23 Appell Equations

Significance

Robots and animals must solve two problems to move: computing the forces or torques needed for a desired motion (inverse dynamics) and determining the motion produced by applied forces (forward dynamics), which are usually modeled separately. We show that both problems can be expressed using a single scalar function from classical mechanics, Appell acceleration energy. A neural network trained so that the derivative of this learned function matches reference joint torques performs *inverse dynamics*. The same network then computes *forward dynamics* by minimizing an objective built from the learned energy landscape, without retraining. This framework provides a unified representation for prediction and control in both neuroscience and robotics.

Abstract

Biological and robotic systems must solve two related computations to move: inverse dynamics, which determines the forces or torques needed to produce a desired movement, and forward dynamics, which computes the resulting accelerations from applied forces or torques. Although these computations are coupled by the same equations of motion, they are usually estimated or implemented as distinct inverse and forward mappings, in both model-based and data-driven formulations. This separation can obscure the shared structure that constrains both problems. Here, we present the Appellian Neural Network, ANNet, a physics-informed neural network that places both computations on a common learned representation by learning a single scalar quantity from classical mechanics—Appell's acceleration energy. The network maps kinematic state and candidate accelerations to this scalar function, and inverse dynamics is obtained by differentiating the learned energy function with respect to acceleration to recover joint torques. Forward dynamics is then calculated without retraining by embedding the same learned energy landscape in an optimization objective whose unconstrained minimum satisfies the Gibbs–Appell equation. The resulting accelerations are integrated forward in time. We evaluate ANNet on a double pendulum paradigm. In trials unseen by the network during training, inverse and optimization-based forward simulations are real-time

accurate. Our results provide a first-principles route for using a single learned representation to support both prediction and control.

Introduction

The ability of biological systems to generate a vast repertoire of precise and coordinated movements, from the rhythmic axial undulations of the lamprey^{1–3} to the intricate dexterity of the human hand^{4,5}, is a hallmark of animal life. Underlying this remarkable capability is the computational challenge of managing the complex physics of movement^{6,7}. To produce any goal-directed action, the nervous system must compute the complex, nonlinear relationship between the forces or torques generated by muscles and the resulting motion of the body's articulated segments^{8–11}. This process involves transforming a desired motor outcome into the specific pattern of muscle activations required to achieve it, a task complicated by the body's intersegmental mechanics and its interaction with a variable environment^{12,13}.

To solve the complex physics of movement, nervous systems employ two fundamental strategies: feedback and feedforward control^{14,15}. Feedback control is a reactive process that uses continuous streams of sensory information primarily from vision^{16–18}, touch¹⁹, and proprioception²⁰ to detect deviations from an intended movement and generate real-time corrections. This mechanism is essential for maintaining accuracy and compensating for unexpected perturbations²¹. However, its effectiveness is limited by significant time delays inherent in sensory processing and neural transmission^{22,23}, which constrain how fast errors can be corrected during fast movements. To reduce dependence on delayed sensory corrections, organisms rely heavily on feedforward control, a predictive strategy that uses prior experience and internal knowledge of the body's dynamics to generate motor commands in advance^{24,25}. By anticipating the required forces, feedforward control enables rapid, coordinated actions that would be impossible under feedback control alone, making it a key component of skilled movement²⁶. Optimal motor performance is widely thought to arise from the seamless integration of both predictive feedforward planning and reactive feedback correction^{27,28}.

Central to the concept of feedforward control is the hypothesis that the brain constructs and maintains internal models—neural representations that mimic the input-output characteristics of the motor system and the environment^{7,29,30}. These models are generally categorized into two complementary types: forward and inverse models³¹. A forward model predicts the sensory consequences of a motor command. Given the current state of a limb and a set of motor commands, it simulates the causal flow of dynamics to predict the limb's next state (e.g., its future position and velocity). This predictive capacity is crucial for overcoming sensory delays²², distinguishing self-generated sensations from external ones³², and providing a rapid estimate of the body's state for online error correction³³. Conversely, an inverse model performs the opposite transformation. It computes the motor commands required to achieve a desired sensory outcome or state transition^{34,35}. By inverting the system's dynamics, an inverse model functions as a controller, enabling the brain to generate the appropriate feedforward commands to execute a planned movement³⁶.

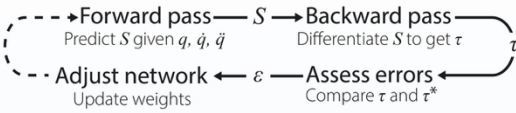
A central and long-standing debate in motor neuroscience is whether these forward and inverse models represent separate computational processes, likely implemented in distinct neural circuits^{31,37}, or whether they could arise from a single, unified neural substrate^{38–40}. The prevailing view has treated them as separate, largely for mathematical reasons. Forward dynamics maps the current state and applied torques to the resulting acceleration ($\ddot{q}$), while inverse dynamics maps the current state and a desired acceleration to the required torques (τ)⁴¹. These are different input-output mappings that, in conventional machine learning, would necessitate different models or networks. This separation is supported by evidence of functional dissociations in neurological disorders and experimental findings suggesting distinct neural correlates for predictive and control-related signals^{42–44}. However, arguments for a more unified architecture have been advanced, often on the grounds of efficiency^{45,46}. Maintaining two separate, complex models for the same physical system could be metabolically and computationally costly, whereas a unified model could leverage shared representations to learn more efficiently. Recent findings suggest that some brain regions, such as the cerebellum and parietal cortex, are engaged during tasks thought to require both model types, leaving open the possibility of a shared substrate⁴⁷.

In this paper, we demonstrate how a single neural network can function as a part of both a forward and an inverse dynamics model. Our approach is grounded in the Gibbs-Appell formulation of analytical mechanics and Gauss's principle of least constraint, which reformulates system dynamics as an optimization problem. This perspective reveals that a single scalar function—Appell acceleration energy—encodes the complete dynamics of a mechanical system. We demonstrate that a physics-informed '*Appellian*' neural network (ANNNet) can be trained to approximate this scalar energy function. For inverse dynamics, the network learns by minimizing the error between the analytical derivative of its scalar output (with respect to acceleration) and the true joint torques, a process enabled by automatic differentiation (see Fig. 1). For forward dynamics, the same trained network is repurposed without any modification. By embedding the learned energy landscape into an optimization objective, the system solves for the accelerations that satisfy the physical principle of least constraint at each time step. Our central hypothesis is that a neural network trained to solve inverse dynamics by learning the scalar Gibbs-Appell function encodes the complete dynamics—encompassing inertial, Coriolis, and gravitational terms—necessary and sufficient to solve forward dynamics. We test this hypothesis using a simulated double pendulum and show that the network, trained exclusively on inverse dynamics data, can accurately predict the system's forward evolution faster than in real time. This work provides a first-principles basis for the unification of forward and inverse internal models within a single computational architecture.

Inverse Dynamics

1. Train the network

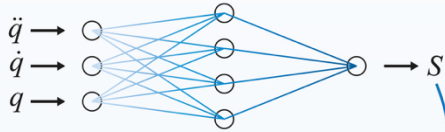
Approximate Appell's function S by minimizing the error between its derivatives and reference torques τ^* :



simulation

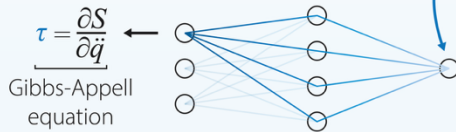
2. Forward pass

Evaluate the trained network to map the state and acceleration inputs $(q, \dot{q}, \ddot{q})$ to the scalar S :



3. Backward pass

Compute torques τ by extracting the gradient of S with respect to acceleration:

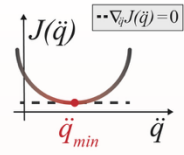


Forward Dynamics

1. Define objective

Minimum of cost function $J(\ddot{q})$ satisfies Gibbs-Appell equation:

$$\nabla_{\ddot{q}} J(\ddot{q}) = \frac{\partial S}{\partial \ddot{q}} - \tau^* = 0$$



$$J(\ddot{q}) = \int \left(\frac{\partial S}{\partial \ddot{q}} - \tau^* \right) d\ddot{q} = S(q, \dot{q}, \ddot{q}) - \tau^* \ddot{q} + \dots$$

simulation

2. Solve for acceleration

Find the acceleration $\ddot{q}_{min}$ that minimizes the cost $J(\ddot{q})$:

$$\ddot{q}_{min} = \underset{\ddot{q}}{\operatorname{argmin}} \left[\underbrace{S(q, \dot{q}, \ddot{q})}_{\substack{\text{initial conditions} \\ \text{network from} \\ \text{inverse dynamics step}}} - \underbrace{\tau^* \ddot{q}}_{\text{regularization term}} + \dots \right]$$

3. Step forward

Integrate found acceleration $\ddot{q}_{min}$ to get the next state:

$$q_{t+\Delta t}, \dot{q}_{t+\Delta t} \leftarrow \int \ddot{q}_{min} dt$$

Fig. 1 | A single neural network trained to predict the Appell function $S(q, \dot{q}, \ddot{q})$ is shared between the inverse and forward dynamics pipelines, eliminating the need to train problem-specific models.

Left: A network learns Appell function, $S(q, \dot{q}, \ddot{q})$, by minimizing the error between its analytical derivative and reference torques, τ^* from analytical equations. In the simulation phase, the forward pass evaluates the network to map kinematic states $(q, \dot{q})$ and accelerations $(\ddot{q})$ to the scalar S , while the backward pass computes torques (τ) for each joint by extracting the derivative of S with respect to acceleration $(\frac{\partial S}{\partial \ddot{q}})$ via the Gibbs-Appell equation. Right: Optimization-based forward dynamics solver. A cost function, $J(\ddot{q})$, is formulated such that its unconstrained minimum analytically satisfies the Gibbs-Appell equation. To solve for acceleration, we isolate $\ddot{q}_{min}$ that minimizes cost $J(\ddot{q})$ —using the learned energy landscape from the inverse step alongside a temporal regularization term—before integrating the resultant acceleration to advance the state variables forward in time $(q_{t+\Delta t}, \dot{q}_{t+\Delta t})$.

Materials and Methods

Analytical formulation of system dynamics

Analytical dynamics of the system were formulated using Gauss's principle of least constraint⁴⁸. According to this principle, a constrained mechanical system evolves so that its actual accelerations deviate as little as possible from the unconstrained accelerations produced by applied active forces. Mathematically, this is achieved by minimizing the mass-weighted constraint function Z over the space of admissible accelerations:

$$Z = \frac{1}{2} \sum_i m_i \left(\ddot{\mathbf{r}}_i - \frac{\mathbf{F}_i}{m_i} \right)^2$$

where m_i represents the mass of the i -th object, $\ddot{\mathbf{r}}_i$ its actual acceleration, and $\mathbf{F}_i$ the applied active force. Expanding Z separates the function into three components:

$$Z = \frac{1}{2} \sum_i m_i \ddot{\mathbf{r}}_i^2 - \sum_i \mathbf{F}_i \ddot{\mathbf{r}}_i + \frac{1}{2} \sum_i \frac{\mathbf{F}_i^2}{m_i}$$

The first term, which is quadratic in acceleration, constitutes the Appell acceleration energy S ^{49,50}. The second term is linear in acceleration, representing the work-like contribution of active forces. The third term is independent of acceleration and is determined exclusively by the system's masses and the applied forces (e.g., gravity). Because these applied forces are determined by the system's current position and velocity rather than its instantaneous acceleration, this third term acts as a mathematical constant during the minimization process.

To find the minimum of the constraint function over the space of admissible accelerations, Z must be minimized with respect to the system's independent generalized accelerations $\ddot{q}_j$, where j is the counter of the degrees of freedom. This requires setting the partial derivative of Z with respect to each $\ddot{q}_j$ to zero:

146
$$\frac{\partial Z}{\partial \ddot{q}_j} = 0$$

147 Because the third term in the expansion of Z is a constant with respect to $\partial \ddot{q}_j$, its derivative vanishes.

148 Therefore, minimizing Z is equivalent to minimizing S , and the condition becomes:

149
$$\frac{\partial S}{\partial \ddot{q}_j} - \sum_i \mathbf{F}_i \cdot \frac{\partial \ddot{\mathbf{r}}_i}{\partial \ddot{q}_j} = 0$$

150 To evaluate the remaining sum, we apply the fundamental kinematic identity that relates Cartesian

151 accelerations to generalized coordinates (a.k.a. 'cancellation of dots'): $\frac{\partial \ddot{\mathbf{r}}_i}{\partial \ddot{q}_j} = \frac{\partial \mathbf{r}_i}{\partial q_j}$. Substituting this identity

152 into the minimization condition gives:

153
$$\frac{\partial S}{\partial \ddot{q}_j} = \sum_i \mathbf{F}_i \cdot \frac{\partial \mathbf{r}_i}{\partial q_j}$$

154 By definition, the right side of this expression constitutes the generalized active force Q_j . Substituting this

155 yields the final Gibbs-Appell equations of motion⁵¹:

156
$$\frac{\partial S}{\partial \ddot{q}_j} = Q_j$$

157 **Derivation of the double pendulum dynamics**

158 We applied analytical derivations from above to model the dynamics of a double pendulum, representing it

159 as two lumped point masses, m_1 and m_2 , within an inertial Cartesian frame. The physical connections

160 between these masses (specifically, the rigid pendulum links of lengths l_1 and l_2) act as holonomic

161 constraints that restrict the system's motion. If we were to track the masses using Cartesian coordinates,

162 we would have to solve a complex system of differential-algebraic equations to continuously enforce these

163 fixed link lengths at every time step. Instead, we parameterized the system using a minimal set of

164 independent generalized coordinates, $\mathbf{q} = [\theta_1, \theta_2]^T$, corresponding to the joint angles. Because these

angles entirely define the system’s configuration, a forward kinematic mapping from $\mathbf{q}$ to the Cartesian position vectors $\mathbf{r}_i$ intrinsically satisfies the geometric constraints.

Next, taking the second time derivative of these position vectors yielded the Cartesian accelerations $\ddot{\mathbf{r}}_i$ as explicit functions of the system’s joint angles, velocities, and accelerations $(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$. The Appell acceleration energy S was then constructed as half the mass-weighted sum of the squared Cartesian accelerations:

$$S = \frac{1}{2}(m_1 \ddot{\mathbf{r}}_1^2 + m_2 \ddot{\mathbf{r}}_2^2)$$

Because the system evolved solely under the influence of gravity, the active generalized forces driving the motion corresponded exclusively to the gravitational joint torques, $\tau_j \equiv -G(q_j)$. As dictated by the Gibbs-Appell formulation⁵², the physical system balances these active forces against the partial derivatives of the acceleration energy with respect to the generalized accelerations. Equating these terms yields the final equations of motion:

$$\frac{\partial S}{\partial \ddot{q}_j} = \tau_j$$

Numerically integrating these ordinary differential equations generated the time-series datasets required for subsequent network training, testing, and validation.

Synthetic data generation and standardization

To generate the training dataset, we sampled kinematic states from uniform distributions: joint angles $q_j \in [-\pi/2, \pi/2]$ rad, velocities $\dot{q}_j \in [-5, 5]$ rad/s, and accelerations $\ddot{q}_j \in [-20, 20]$ rad/s², for the links $j \in \{1, 2\}$. These bounds were chosen to cover physically realistic states that the system could experience under gravity (recognizing that independent, coordinate-wise uniform sampling defines an axis-aligned region rather than a rotationally invariant distribution in the six-dimensional state space). For each sampled state vector $\mathbf{z} = [\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}]^\top$, we computed the Appell acceleration energy S and the torques $\boldsymbol{\tau}$ using the analytical Gibbs-Appell formulation defined above. We standardized the input states $\mathbf{z}$ and the acceleration energy S

to zero mean and unit variance, denoting these normalized, dimensionless variables as $\mathbf{z}^*$ and S^* . The target torques $\boldsymbol{\tau}$ remained unnormalized. Since subsequent computational differentiation would evaluate the gradient of S^* with respect to the dimensionless accelerations $\ddot{\mathbf{q}}^*$, we computed and stored a chain-rule scaling vector, $\sigma_S/\boldsymbol{\sigma}_{\ddot{\mathbf{q}}}$. Here, the scalar σ_S and the two-element vector $\boldsymbol{\sigma}_{\ddot{\mathbf{q}}}$ denote the sample standard deviations of the energy and the accelerations across the dataset, respectively, and provide the transformation required to map the dimensionless partial derivatives back to the physical domain, ensuring that $(\sigma_S/\boldsymbol{\sigma}_{\ddot{\mathbf{q}}}) \odot (\partial S^*/\partial \ddot{\mathbf{q}}^*) = \boldsymbol{\tau}$, where $\odot$ denotes the element-wise product.

Neural network architecture

We designed a feedforward artificial neural network to map the normalized state vector $\mathbf{z}^* \in \mathbb{R}^6$ to the dimensionless Appell acceleration energy $S^* \in \mathbb{R}$. The finalized architecture, determined via the hyperparameter optimization (see *Hyperparameter optimization and architectural search*), comprised a multilayer perceptron with four hidden layers containing 225, 61, 91, and 99 computational nodes, respectively. We applied Gaussian Error Linear Unit (GELU) activation functions to each hidden layer⁵³ and terminated the network with a single linear node to output the scalar acceleration energy. Batch normalization was not applied within the layers; instead, the six input coordinates were standardized using fixed dataset-level statistics before training, and the corresponding chain-rule scaling was used to convert derivatives in normalized coordinates back to physical torque units.

Physics-informed loss formulation

To train the network, we formulated a physics-informed loss function based on the Gibbs-Appell equations. Rather than computing a loss on the scalar acceleration energy output directly, we evaluated the network’s ability to predict joint torques. We chose this gradient-based supervision for two reasons. Theoretically, minimizing the error on a scalar function does not guarantee that a neural network will accurately capture the function’s gradients, which are required to represent the actual equations of motion. Practically, deriving the analytical Appell acceleration energy is cumbersome for complex, high-DOF systems, whereas joint torques are standard physical quantities that can be obtained readily via sensors or inverse dynamics

solvers. Notably, the relevant constraint is not global recovery of the scalar acceleration-energy surface, but accuracy of the scaled input-gradient that enters the Gibbs–Appell equations. Our loss therefore supervises this derivative quantity directly, consistent with Sobolev training, where derivative information is incorporated into the learning objective⁵⁴.

During each forward pass, we computed the partial derivatives of the network’s output S_{net}^* with respect to the normalized input accelerations $\ddot{\mathbf{q}}^*$ using automatic differentiation. We multiplied these dimensionless gradients by the precomputed scaling vector $\sigma_S/\sigma_{\ddot{\mathbf{q}}}$ to predict torques, $\boldsymbol{\tau}_{\text{pred}}$. We defined the optimization objective as the mean squared error between $\boldsymbol{\tau}_{\text{pred}}$ and the ground-truth joint torques $\boldsymbol{\tau}$ across the batch size $B = 64$:

$$\mathcal{L} = \frac{1}{B} \sum_{k=1}^B \left\| \frac{\sigma_S}{\sigma_{\ddot{\mathbf{q}}}} \odot \frac{\partial S_k^*}{\partial \ddot{\mathbf{q}}_k^*} - \boldsymbol{\tau}_k \right\|^2,$$

where k is the index variable for the individual data samples within a given mini-batch (a set of samples processed together). This formulation constrained the network to learn the underlying scalar energy space exclusively through the supervision of its vector field gradients.

Hyperparameter optimization and architectural search

To identify the optimal network architecture and training hyperparameters, we conducted a systematic hyperparameter search⁵⁵. We evaluated candidate configurations on an independent dataset of 10,000 kinematic samples (chosen to densely span the underlying kinematic phase space), partitioned into an 80% training set and a 20% testing set, applying the previously described standardization and chain-rule scaling transformations. The architectural search space included a discrete range of 1 to 10 hidden layers (with 1 to 256 nodes per layer) and a categorical selection of activation functions: GELU, Tanh, Softplus, ELU, and ReLU. Finally, the training learning rate was sampled from a continuous log-uniform distribution between 10^{-8} and 10^{-1} .

234 The search space was sampled across 500 independent trials, each representing a distinct parameter set.
235 We evaluated each set of parameters for 100 epochs, using the physics-informed loss $\mathcal{L}$ as the minimization
236 objective. To allocate computational resources toward converging models, we employed a median pruning
237 algorithm that terminated trials whose performance was worse than the historical median validation loss
238 after a ten-epoch warmup phase. Of the 500 initiated trials, the pruner terminated 319.

239 The algorithm converged on an optimal architecture of 225, 61, 91, and 99 nodes for the first, second, third,
240 and fourth hidden layers, respectively, alongside GELU activations and a learning rate of 1.13×10^{-3} . This
241 optimal configuration served as the final architecture for all subsequent experiments.

242 **Experimental setup and optimization**

243 To evaluate the data requirements of the architecture, independent models of the same architecture were
244 trained on nine datasets doubling in size from 1,000 to 256,000 total vectors $\mathbf{z}^*$. Each dataset was
245 partitioned using the established 80% training and 20% testing split. The network weights were optimized
246 using the Adam algorithm⁵⁶ with the optimal learning rate described in the previous section. Each model
247 was trained for 3,000 epochs. Continuous training loss and periodic validation loss were recorded to
248 evaluate convergence, and inference timing was measured to assess the computational viability of the
249 trained internal models. The model instance yielding the minimum validation error for each dataset size
250 was selected for subsequent analysis. Thus, downstream analyses used the checkpoint with the minimum
251 periodic validation error, rather than the weights from the final training epoch.

252 **Generation of ground-truth trajectories via forward dynamics**

253 The inverse dynamics predictions of the trained networks were evaluated using a fixed test dataset
254 comprising 50 independent temporal trajectories generated through numerical integration. The partial
255 derivatives of the Appell acceleration energy were set equal to the gravitational forces derived from the
256 potential energy $V = m_1 \cdot g \cdot y_1(\theta_1, \theta_2) + m_2 \cdot g \cdot y_2(\theta_1, \theta_2)$, yielding:

$$\frac{\partial S}{\partial \ddot{\mathbf{q}}} = - \frac{\partial V}{\partial \mathbf{q}}$$

We algebraically inverted this system of equations to isolate the generalized accelerations $\ddot{\mathbf{q}}$ as explicit functions of the current state (see *Forward dynamics equations in the Supplementary information section*). To enable numerical integration, we reformulated the resulting dynamics as a system of continuous first-order ordinary differential equations. To establish the initial conditions, we sampled the joint angles uniformly from the interval $[-\pi/3, \pi/3]$ rad and set the initial angular velocities to 0 rad/s, which ensured that the subsequent motion was driven exclusively by gravitational forces. This sampling process was repeated to generate initial conditions for 50 independent trajectories. Finally, we numerically integrated this system of equations over a five-second duration using a fourth-order Runge-Kutta scheme with a fixed time step of 10^{-4} s (see ^{57,58} for similar solver configuration).

Neural network-based inverse dynamics

We deployed the trained models to predict the inverse dynamics of the 50 unseen ground-truth trajectories. For each model, we normalized the kinematic state vectors of the test set using the mean and standard deviation parameters derived from its specific training distribution. The network mapped these standardized inputs to predict the dimensionless acceleration energy S^* . We evaluated the partial derivatives of this scalar output with respect to the normalized input accelerations via automatic differentiation. We then multiplied these dimensionless gradients by the model-specific scaling vector to recover the predicted physical joint torques:

$$\boldsymbol{\tau}_{\text{pred}} = (\sigma_s / \sigma_{\ddot{\mathbf{q}}}) \odot \left(\frac{\partial S^*}{\partial \ddot{\mathbf{q}}^*} \right)$$

To quantify the predictive accuracy of each model, we computed the RMSE between $\boldsymbol{\tau}_{\text{pred}}$ and the analytical ground-truth, evaluated independently for each joint across all simulated time steps.

278 Forward dynamics via optimization

279 To solve the forward dynamics problem without requiring a distinct forward model, we repurposed the
 280 trained inverse dynamics network and formulated the derivation of joint accelerations as an unconstrained
 281 numerical optimization task. Following the Gibbs-Appell formulation, we established the fundamental
 282 equation of motion by equating the gradient of the acceleration energy with respect to the generalized
 283 accelerations to the generalized external forces. For the simplified double-pendulum system considered
 284 here, we set gravity as the only external generalized force; notably, this choice defines the test problem
 285 and is not a restriction of the network or optimization formulation. In systems with additional applied torques
 286 or external forces, these terms can be added to the generalized-force vector. Denoting gravitational
 287 contribution as $\boldsymbol{\tau} \equiv -\mathbf{G}(\mathbf{q})$, we obtained the governing equation of zero balance:

$$288 \quad \nabla_{\ddot{\mathbf{q}}} S(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) + \mathbf{G}(\mathbf{q}) = 0,$$

289 where $\nabla_{\ddot{\mathbf{q}}} = \frac{\partial}{\partial \ddot{\mathbf{q}}}$. To resolve this equation algorithmically, we sought a scalar objective function $J(\ddot{\mathbf{q}})$ such that
 290 its unconstrained minimization (characterized by the stationary condition $\nabla_{\ddot{\mathbf{q}}} J(\ddot{\mathbf{q}}) = 0$) mathematically
 291 satisfies this balance of torques. We constructed $J(\ddot{\mathbf{q}})$ by taking the integral of the equation of zero balance
 292 with respect to the acceleration vector $\ddot{\mathbf{q}}$:

$$293 \quad J(\ddot{\mathbf{q}}) = \int \left(\nabla_{\ddot{\mathbf{q}}} S(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) + \mathbf{G}(\mathbf{q}) \right) d\ddot{\mathbf{q}}$$

294 Because the inertial term is the gradient of the acceleration energy with respect to $\ddot{\mathbf{q}}$, integrating it over
 295 acceleration space recovers $S(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$, up to a term that may depend on $\mathbf{q}$ and $\dot{\mathbf{q}}$ but not on $\ddot{\mathbf{q}}$. The
 296 gravitational vector $\mathbf{G}(\mathbf{q})$ is independent of acceleration, so its integral with respect to $\ddot{\mathbf{q}}$ is the inner product
 297 $\mathbf{G}(\mathbf{q})^\top \ddot{\mathbf{q}}$. Combining these terms gives the objective function of form:

$$298 \quad J(\ddot{\mathbf{q}}) = S(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) + \mathbf{G}(\mathbf{q})^\top \ddot{\mathbf{q}} + \mathcal{C}(\mathbf{q}, \dot{\mathbf{q}})$$

299 The aggregated integration constant $\mathcal{C}(\mathbf{q}, \dot{\mathbf{q}})$ *does not* depend on the optimization variable $\ddot{\mathbf{q}}$. Therefore,

$$arg \min_{\ddot{\mathbf{q}}} [J(\ddot{\mathbf{q}}) + C(\mathbf{q}, \dot{\mathbf{q}})] = arg \min_{\ddot{\mathbf{q}}} J(\ddot{\mathbf{q}}),$$

so, omitting $C(\mathbf{q}, \dot{\mathbf{q}})$ during the computation of the acceleration that minimizes the cost ($\ddot{\mathbf{q}}_{min}$) does not change the minimizing acceleration. As such, while retained for analytical correctness, term $C(\mathbf{q}, \dot{\mathbf{q}})$ was omitted during the computational implementation.

To formulate the final objective function for simulation, we replaced the analytical S with the network-predicted Appellian S_{net} . Because the network outputted a dimensionless, standardized scalar S_{net}^* , we mathematically recovered the physical acceleration energy by applying the inverse Z-score transformation, $S_{net} = S_{net}^* \sigma_S + \mu_S$, using the precomputed dataset statistics. This denormalization ensured dimensional consistency across the objective function, to which we added a temporal regularization term:

$$J(\ddot{\mathbf{q}}) = S_{net}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) + \mathbf{G}(\mathbf{q})^T \ddot{\mathbf{q}} + \lambda \|\ddot{\mathbf{q}} - \ddot{\mathbf{q}}_{prev}\|^2$$

where $\ddot{\mathbf{q}}_{prev}$ denotes the acceleration from the preceding timestep, and $\lambda = 0.1$ is the regularization coefficient.

We minimized the objective function at each timestep using a two-stage solver, with optimal parameters determined via grid search (see *Performance optimization of the optimization-based forward dynamics solver section*). In the first stage, a coarse global search evaluated the cost function across 16 candidate acceleration vectors. This candidate pool comprised the zero vector, the acceleration from the previous timestep, and 14 vectors randomly sampled from a uniform distribution bounded by ± 15 rad/s².

In the second stage, the candidate yielding the minimum cost was used to initialize a gradient-based local refinement. We executed 10 optimization steps using the Adam algorithm (learning rate = 0.1) to converge on the final minimizer. This scheme solved the forward dynamics problem by mapping kinematic states to accelerations strictly through the gradient landscape of the learned energy field, circumventing the need for an independent forward model.

Performance optimization of the optimization-based forward dynamics solver

The accuracy and efficiency of this two-stage solver depend heavily on the scale of the candidate pool and the depth of the local gradient refinement. To systematically identify the optimal solver configuration, we performed a grid search across 336 distinct parameter sets. Our search evaluated three parameters: the number of sampled acceleration vectors (N), the uniform acceleration sampling bound (B), and the number of Adam refinement steps (A) (Table 1). We evaluated each configuration over 50 trajectories with 200 closed-loop state updates per trajectory, yielding 10,000 updates per configuration.

Table 1. Parameter search space for the forward-dynamics solver.

Parameter	Values tested
Sampled acceleration vectors, N	16, 32, 64, 128, 256, 512, 1024
Uniform sampling bound, B (rad/s ²)	+/-5, +/-10, +/-15, +/-20, +/-25, +/-30, +/-35, +/-40
Adam refinement steps, A	0, 1, 2, 5, 10, 15

Solver performance was assessed by joint angular accuracy and computational latency. The primary accuracy metric was the mean absolute error (MAE) across both joints, calculated per state update as:

$$MAE = \frac{|q_{1,true} - q_{1,pred}| + |q_{2,true} - q_{2,pred}|}{2},$$

where $q_{1,true}$ and $q_{2,true}$ are the ground-truth angles for joint 1 and joint 2, and $q_{1,pred}$ and $q_{2,pred}$ are the corresponding predicted angles. This value was averaged across the 10,000 updates.

To assess computational speed, we recorded the full state-update time rather than isolated neural-network inference time. This metric accounts for optimizer execution, candidate evaluation, network forward passes, Adam refinement, numerical integration, tensor operations, and loop overheads. Given a simulation timestep of 2 ms, a configuration was defined as real-time capable if its mean update time was ≤ 2.0 ms.

Closed-loop simulation

The predictive accuracy of the optimization-based forward dynamics solver was evaluated by conducting closed-loop forward simulations across a distribution of randomly initialized kinematic states. For each initial state, the system was integrated forward in time using a semi-implicit Euler integration scheme ($\Delta t = 2$ ms), chosen to maintain the numerical stability of the first-order update. At each temporal increment, the solver assessed the current state variables ($\mathbf{q}, \dot{\mathbf{q}}$) and computed the resultant acceleration vector, $\ddot{\mathbf{q}}$. To stabilize the temporal regularization term at the onset of each simulation, a five-iteration stabilization loop was applied at $t = 0$ s, with the optimizer's output recursively fed back to establish the boundary condition for $\ddot{\mathbf{q}}_{prev}$.

Integration drift was quantified by generating corresponding ground-truth trajectories for identical initial states using an explicit fourth-order Runge-Kutta integration scheme ($\Delta t = 2$ ms) (see ⁵⁹ for similar solver configuration). The continuous state evolution was recorded, and the RMSE was computed between the simulated and ground-truth angles, velocities, and accelerations across the temporal horizon for both joints.

Statistical analysis

All statistical analyses were performed using SciPy⁶⁰ and statsmodels⁶¹, while neural network computations and automatic differentiation were executed via PyTorch⁶². No trials were excluded. We evaluated the assumption of normality for all empirical distributions using the Shapiro-Wilk test ($\alpha = 0.05$). Because distributions for temporal latencies and RMSE deviated from normality, non-parametric tests were used for hypothesis testing. Two-sided Mann-Whitney U tests were used for pairwise RMSE-distribution comparisons, whereas one-sided Mann-Whitney U tests were used for directional latency comparisons when the prespecified hypothesis was that the optimization-based forward dynamics solver required longer execution time than the traditional, numerical solver.

For the inverse-dynamics evaluation, predictive accuracy was assessed by calculating the torque RMSE at the first and second joints across 50 ground-truth numerical integration trajectories. To map the statistical

effect of training dataset volume on predictive accuracy, we performed pairwise comparisons of the RMSE distributions across dataset sizes using two-sided Mann-Whitney U tests. These pairwise comparisons were corrected for multiple comparisons using the Benjamini-Hochberg false discovery rate (FDR) procedure.

To quantify computational efficiency and eliminate hardware biases, models were evaluated in a randomized, interleaved order with hardware operations synchronized immediately before and after execution. Timing summaries distinguish three distinct quantities. Network-forward time isolates the duration required for neural-network forward evaluations. Solver-call time encompasses candidate sampling, candidate scoring, and Adam refinement for a single optimizer call. Full-update time is the total wall-clock rollout time divided by the number of simulated state updates. Because the full-update metric includes the complete simulation loop overhead, it serves as the most conservative timing measure and was exclusively used to evaluate real-time performance. Differences in inverse-model inference latency across dataset sizes were evaluated using a Kruskal-Wallis H-test.

For the forward-dynamics evaluation, solver accuracy was assessed by computing the RMSE for joint angles, velocities, and accelerations. We compared the spatial evolution of the closed-loop forward simulations against analytical ground truth over fifty 5-second trajectories. Finally, to evaluate computational viability, we compared the execution latencies of the optimization-based solver against a standard numerical ODE solver using the full-update time metric, with statistical significance determined by a one-sided Mann-Whitney U test testing whether optimization-based solver execution time exceeded analytical ODE execution time.

Hardware specifications

All computational experiments, including dataset generation, network training, and dynamics simulations, were performed locally on an Apple M4 system-on-a-chip equipped with 32 GB of unified memory, running macOS Tahoe (v.26.3.1). Computations were executed exclusively on the CPU cores, without utilizing the integrated Metal Performance Shaders (MPS) for hardware acceleration.

Results

Hyperparameter search optimized torque recovery from learned Appell energy

We selected the network architecture using inverse-dynamics accuracy as the performance criterion. Each candidate network received the standardized six-dimensional input vector $\mathbf{z}^*$ and returned a scalar estimate of Appell acceleration energy, S^* . To evaluate each candidate, we differentiated this scalar output with respect to the acceleration inputs and rescaled the derivative to obtain predicted joint torques. We therefore ranked the 500 hyperparameter trials by torque RMSE, directly testing the quantity required for inverse dynamics. Global sensitivity analysis showed that performance was most strongly affected by the width of the first hidden layer and the learning rate, which accounted for 51.4% and 25.5% of the variation in RMSE, respectively. Network depth accounted for 18.3%, whereas activation function accounted for 4.8%, indicating smaller but measurable effects (Fig. 2h).

The search revealed clear limits on the hyperparameter values that produced accurate models. Networks trained with learning rates below 10^{-5} did not converge and remained at high torque errors near 43 Nm (Fig. 2a). Lower errors required learning rates near 10^{-3} . Accuracy also depended strongly on the width of the first hidden layer: the best-performing trials used first-layer widths close to the upper end of the search range, 225 nodes (Fig. 2c). In contrast, later hidden layers were less sensitive to width and reached low errors with fewer units (Fig. 2e–g). The lowest torque RMSE was obtained with a four-hidden-layer network, indicating that this depth performed best among the tested architectures (Fig. 2b).

Although the activation function had a smaller overall effect than learning rate or first-layer width, it still influenced the minimum error achieved by each model (Fig. 2d). Networks with GELU activations achieved lower torque RMSE than networks using ReLU, ELU, Softplus, or Tanh. This result is consistent with the derivative-based training objective, because torque predictions depend on the derivative of the learned scalar S^* with respect to acceleration. Across the full search, the best-performing configuration used a learning rate of 1.13×10^{-3} , four hidden layers with 225, 61, 91, and 99 units, and GELU activations (Fig.

2a–g, red markers). We used this architecture for all subsequent analyses. A condensed parallel-coordinate
 representation of the full hyperparameter search is provided in Supplementary Fig. 1.

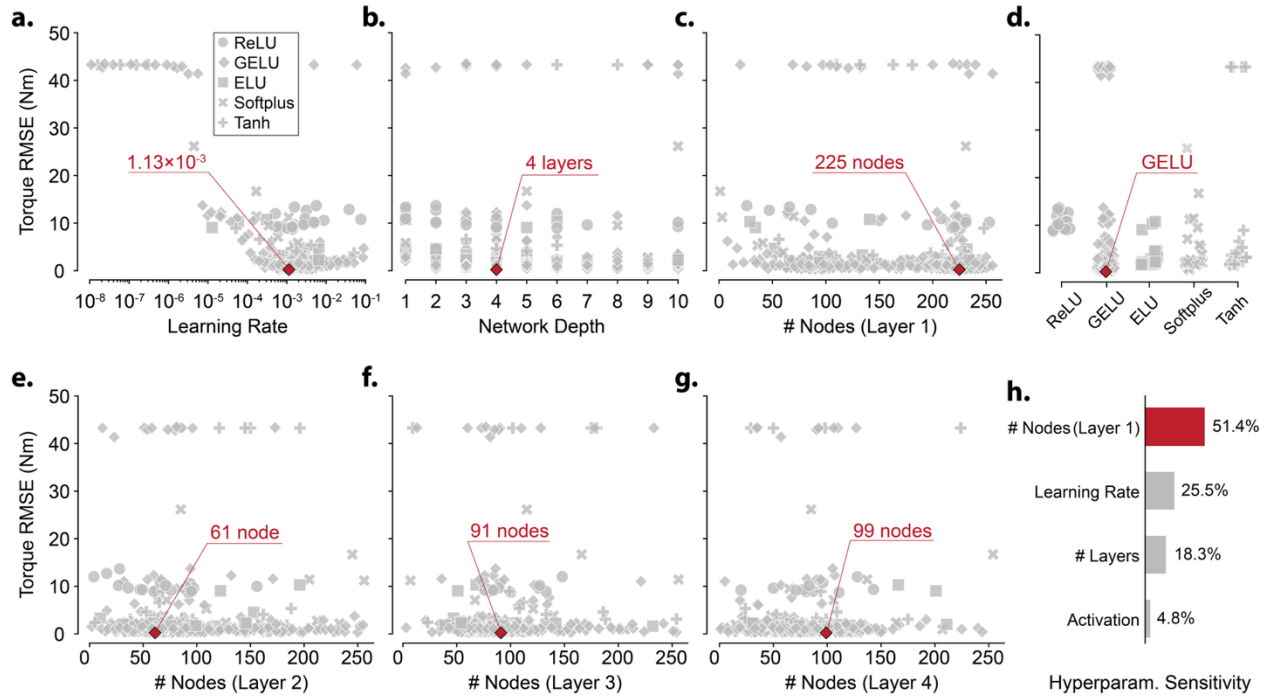


Fig. 2 | Systematic hyperparameter optimization.

a–g, Torque prediction error for individual hyperparameter-optimization trials, reported as root-mean-square error (RMSE; Nm), plotted against learning rate (**a**), network depth (**b**), number of nodes in hidden layer 1 (**c**), activation function (**d**), and number of nodes in hidden layers 2–4 (**e–g**). Each grey symbol represents one evaluated model configuration; symbol shape denotes the activation function, as indicated in **a**. Red diamonds mark the best-performing configuration: learning rate = 1.13×10^{-3} , network depth = 4 layers, GELU activation, and 225, 61, 91 and 99 nodes in hidden layers 1–4, respectively. **h,** Relative hyperparameter sensitivity, showing the proportional contribution of each parameter to variation in optimization performance.

Inverse-dynamics accuracy reached a low-error regime at intermediate dataset size

Increasing the training set generally reduced inverse-dynamics error, but the improvement was not monotonic (Fig. 3). This non-monotonicity is expected under the fixed-architecture protocol used here. The

architecture and learning rate were optimized once on an independent 10,000-sample architecture-search dataset and then held constant for all dataset-size experiments, so each point reflects data scaling within the same model class rather than a separately tuned model. Re-optimizing the architecture for every dataset size would conflate data efficiency with model selection, and the small reversals likely reflect finite held-out-set variation plus interactions between fixed model capacity, learning rate, training duration, and each training set. Across dataset sizes, validation RMSE declined rapidly during early training and then fluctuated within a lower-error regime rather than converging smoothly (Fig. 3a). These gains also came with a steep increase in computational cost, as training time rose from 29.6 s for 1,000 samples to 2.16 h for 256,000 samples (Fig. 3b).

Held-out performance showed a similar scaling pattern. From 1,000 to 16,000 samples, the median RMSE decreased from 0.1523 to 0.0242 Nm for joint 1 and from 0.0800 to 0.0234 Nm for joint 2 (Fig. 3c,e). Beyond this range, additional data produced smaller and less regular gains. Joint 2 error increased slightly between 4,000 and 8,000 samples, and both joints showed a transient increase in error at 32,000 samples before the downward trend resumed at larger dataset sizes. The lowest median errors occurred at 256,000 samples, reaching 0.0189 Nm for joint 1 and 0.0118 Nm for joint 2. Thus, larger datasets improved the best attainable accuracy, but much of the gain had already emerged by the intermediate-data regime. We therefore used the 16,000-sample model for subsequent analyses because it captured most of the accuracy improvement while avoiding the substantially longer training times required at larger dataset sizes.

Consistent with this pattern, the aggregate dependence of error on dataset size was well described by power-law fits to the median RMSE values: $RMSE_{J_1}(n) = 2,1782.3 \cdot n^{-1.75} + 0.0293$, $R^2 = 0.974$ for joint 1; $RMSE_{J_2}(n) = 108.958 \cdot n^{-1.089} + 0.0195931$, $R^2 = 0.921$ for joint 2, where n denotes the number of training samples. RMSE distributions were non-normal at all dataset sizes (Shapiro-Wilk, all $p < 0.05$), so group comparisons were performed using two-sided Mann-Whitney U tests with Benjamini-Hochberg correction shown as chord diagrams in Fig. 3d,f. An equivalent matrix representation of these FDR-corrected pairwise comparisons is provided in Supplementary Fig. 2. Relative to the 1,000-sample model, the 16,000-sample

model significantly reduced error for both joints (joint 1, $p = 8.48 \times 10^{-17}$, $p^* = 7.64 \times 10^{-16}$; joint 2, $p = 2.65 \times 10^{-14}$, $p^* = 2.01 \times 10^{-13}$), supporting its use as an efficient operating point for downstream analyses.

Inference latency was effectively independent of dataset scale. Across 10,000 interleaved executions per model, median latency ranged from $73.29 \mu\text{s}$ to $73.38 \mu\text{s}$, with IQRs of $1.59\text{-}1.71 \mu\text{s}$. Although latency distributions differed statistically across dataset sizes (Kruskal-Wallis, $H = 28.6616$, $p = 3.63 \times 10^{-4}$), the total span of median latencies was only $0.083 \mu\text{s}$, indicating no practically meaningful deployment penalty from training on larger datasets.

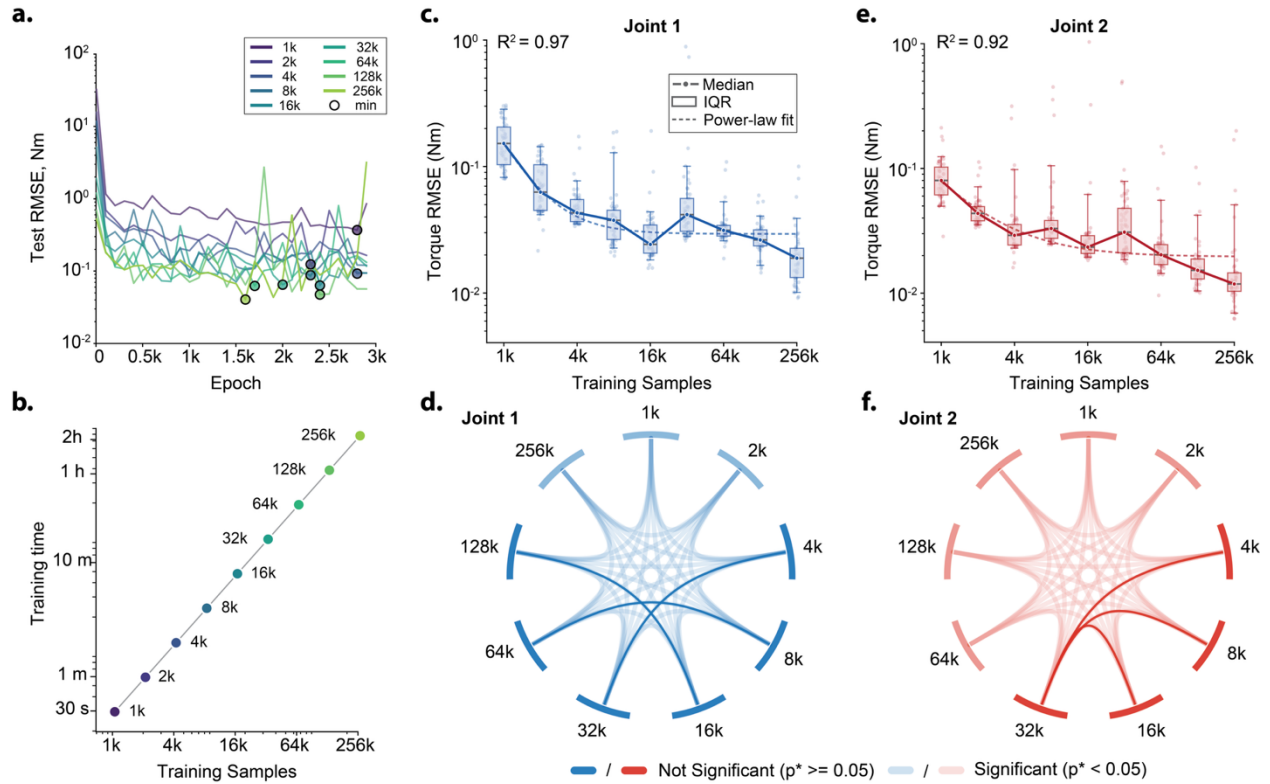


Fig. 3: Dataset-size effects on inverse-dynamics accuracy and training cost.

a, Test RMSE during training for fixed-architecture networks trained for 3,000 epochs on datasets ranging from 1,000 to 256,000 samples. Markers indicate the validation-minimum checkpoint selected for downstream analyses. **b,** Wall-clock training time as a logarithmic function of training-set size. **c,e,** Torque RMSE distributions for joint 1 (**c**) and joint 2 (**e**), evaluated on 50 independent 5-s test trajectories at each dataset size. Axes are logarithmic; points show errors

for individual trajectories; boxes indicate the IQR, centre lines indicate medians, and whiskers indicate the 5th–95th percentile range. Dashed lines show power-law fits to the median RMSE values, with R^2 values indicated. **d,f**, Pairwise comparisons of RMSE distributions across dataset sizes for joint 1 (**d**) and joint 2 (**f**), computed using two-sided Mann–Whitney U tests with Benjamini–Hochberg correction. Link opacity indicates whether the corrected comparison was significant, as defined in the legend.

Forward-solver performance depended mainly on refinement depth and search bound

We next evaluated how to optimally configure the optimization-based forward solver to accurately recover accelerations while maintaining real-time performance. Across 336 solver configurations, accuracy and latency formed a clear Pareto trade-off (Fig. 4). Configurations with little or no Adam refinement were fast but often inaccurate, whereas additional refinement reduced angular error at the cost of longer update times. Across the grid, median angular error decreased from 2.65×10^{-2} rad with no Adam steps to 8.37×10^{-4} rad with 10 Adam steps, while median update time increased from 0.22 ms to 1.73 ms. Increasing refinement to 15 steps produced only a small further reduction in error but raised the median update time above the 2.0-ms real-time limit.

The acceleration sampling bound also affected solver reliability. Moderate bounds produced the lowest-error operating points, whereas very broad bounds generated many high-error configurations, especially when local refinement was limited. The lowest-error configuration used 16 sampled acceleration vectors, a ± 20 rad/s² sampling bound, and 15 Adam steps, reaching a mean absolute angular error of 7.12×10^{-4} rad but requiring 2.37 ms per update. We therefore selected the best real-time-capable configuration: 16 sampled vectors, ± 15 rad/s² bound, and 10 Adam steps. This setting preserved nearly the same accuracy (7.50×10^{-4} rad) while reducing the mean update time to 1.68 ms. A faster balanced setting reduced update time to 0.92 ms with slightly higher error (1.15×10^{-3} rad), whereas the fastest configuration completed updates in 0.13 ms but had substantially larger error (0.397 rad).

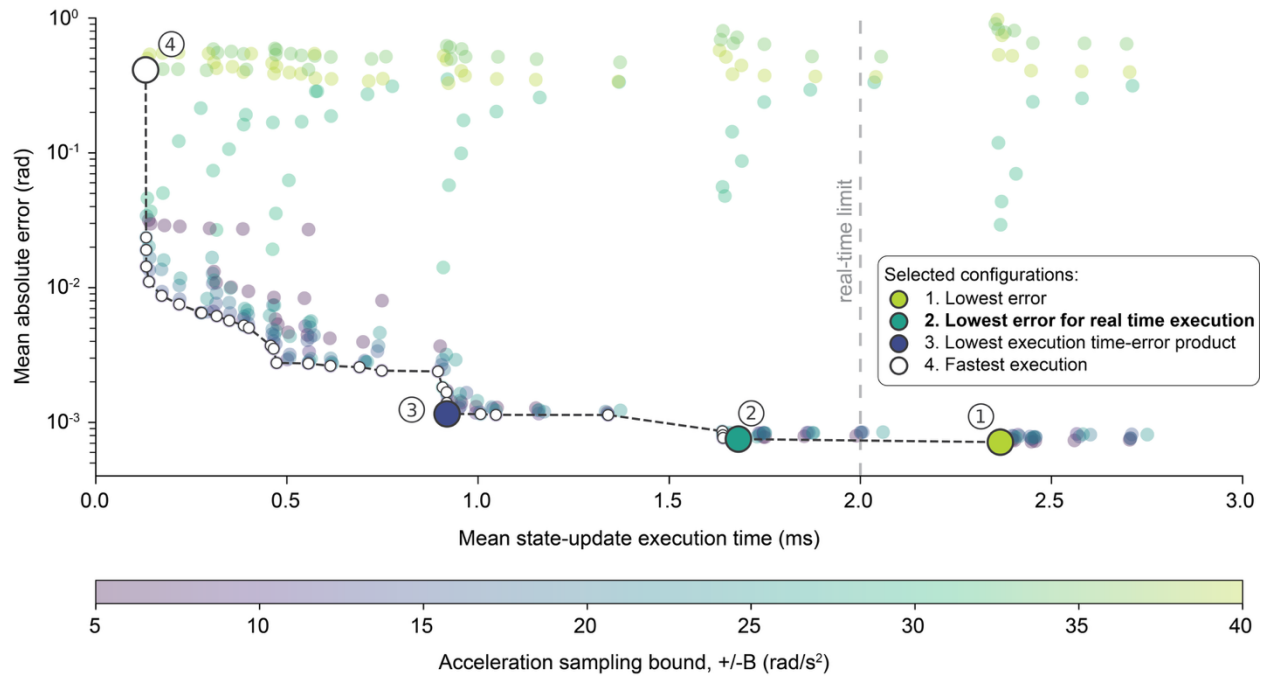


Fig. 4 | Accuracy-latency trade-offs of the optimization-based forward dynamics solver.

Each point denotes one of 336 solver configurations, evaluated over 50 trajectories (10,000 closed-loop state updates). The y axis shows the mean absolute angular error, averaged across the two joints and plotted on a logarithmic scale; the x axis shows the mean state-update execution time. The grid varied the number of sampled acceleration vectors (N), the uniform acceleration bound (B , in $\pm$ rad/s²) and the number of Adam refinement steps (A); only B is encoded visually, from smaller bounds in purple to larger bounds in yellow. The black dashed curve marks the empirical Pareto frontier, and the grey dashed vertical line marks the 2.0-ms real-time limit. Numbered callouts identify the lowest-error configuration (1, red; $N = 16$, $B = \pm 20$, $A = 15$), the selected real-time operating point (2, blue; $N = 16$, $B = \pm 15$, $A = 10$), the minimum execution-error product configuration (3, dark red; $N = 32$, $B = \pm 10$, $A = 5$) and the fastest configuration evaluated (4, open blue outline; $N = 16$, $B = \pm 35$, $A = 0$).

Learned Appell energy landscape enables accurate, real-time forward dynamic simulation

The selected optimization-based neural forward dynamics solver configuration ($N = 16$, $B = \pm 15$ rad/s², $A = 10$) was tested in closed-loop forward simulation. The solver was evaluated on 50 reference trajectories, each lasting 5 s. At each time step, it estimated acceleration by first scoring 16

candidate acceleration vectors within the prescribed bound and then refining the best candidate with 10 Adam steps. The objective combined the learned Appell-energy term, the gravitational work term and a penalty for changes from the previous acceleration estimate.

The optimization-based solver closely followed the reference motion over the 5-s rollouts (Fig. 5a). Median angular RMSEs were 0.00737 rad for joint 1 and 0.0119 rad for joint 2. Median velocity RMSEs were 0.0309 and 0.0677 rad/s, and median acceleration RMSEs were 0.165 and 0.382 rad/s², respectively. Errors accumulated gradually during the rollout, as expected for a closed-loop simulation that repeatedly feeds back its own acceleration estimates, but remained bounded over the full simulation horizon (Fig. 5b).

This accuracy came with a computational cost relative to direct numerical evaluation. For complete 5-s trajectories, the optimization-based solver required a median of 4.04 s, compared with 0.547 s for the numerical solver (Fig. 5c, one-sided Mann-Whitney U test, $U = 2500$, $p = 3.53 \times 10^{-18}$). At the individual update level, however, the solver remained within the 2.0-ms real-time budget: median solver latency was 1.54 ms with an interquartile range of 0.0655 ms, and the mean update time over full rollouts was 1.62 ms (Fig. 5d). Thus, the learned Appell-energy landscape, while remaining slower than the traditional numerical integrator for complete trajectory generation, enabled accurate, real-time-capable forward simulation.

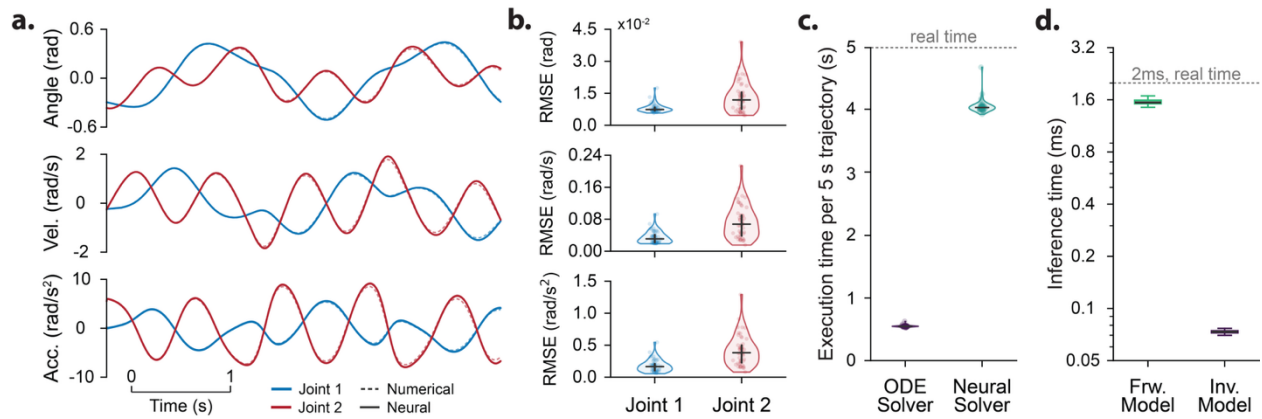


Fig. 5 | Learned acceleration energy landscape enables accurate and fast forward simulation.

a, Representative closed-loop rollouts for the two-joint system, comparing the optimization-based neural forward dynamics solver with the numerical ODE solution for angular position, velocity and acceleration. Blue and red denote joints 1 and 2, respectively; solid and dashed traces denote neural and numerical trajectories. **b**, Trajectory-wise RMSE distributions for position, velocity and acceleration across 50 independent simulations, shown separately for each joint. Points denote individual trajectories, distribution envelopes summarize the spread across simulations, horizontal lines denote medians and vertical bars denote the interquartile range. **c**, Wall-clock execution time required to generate a complete 5 s trajectory using the numerical ODE solver or the optimization-based neural forward dynamics solver. The dotted line denotes the 5 s real-time threshold. **d**, Per-query inference latency for the forward model (optimization-based neural forward dynamics solver step) and the inverse model (automatic differentiation through ANNet). Forward latency was measured for the optimized solver configuration, and inverse latency was measured for the 16k-sample trained network. The y-axis is shown on a logarithmic scale, and the dotted line denotes the 2 ms real-time inference threshold.

Discussion

Across a wide range of scientific and engineering settings, dynamics models are valuable because they can be used either in the inverse direction to compute inputs for control or in the forward direction to predict the consequences of those inputs^{30,63}. In this study, we show that a physics-structured neural representation can achieve high-accuracy inverse dynamics predictions at low inference cost, while also supporting accurate closed-loop forward prediction over multi-second horizons. As such, constraining learning with a

first-principles structure yielded a compact internal model that is simultaneously actionable for control-relevant inverse computations and forward predictions, which is an important step toward developing artificial systems that mimic biological motor coordination.

Network topology and hyperparameters

Hyperparameter optimization indicated that most performance variation in the explored space was driven by initial hidden-layer width and learning rate, rather than deeper architectural choices, with initial hidden-layer width accounting for 51.4% and learning rate accounting for 25.5% of the global sensitivity, compared with 4.8% for activation function choice and 18.3% for depth within one to ten layers. This concentration of sensitivity aligns with observations in physics-informed training more generally, that increasing network size and noise can increase optimization error by making the loss landscape more complex. As a result, convergence can depend strongly on choices that stabilize optimization rather than on incremental representational capacity alone⁶⁴. It is also aligned with reports of diminishing returns beyond width and depth thresholds, where added complexity yields only small performance improvements while increasing computational cost^{65,66}.

The final configuration identified by the search was a multilayer perceptron with four hidden layers of 225, 61, 91, and 99 nodes, GELU activations, and learning rate 1.13×10^{-3} , selected after sampling 500 independent trials with 319 pruned early by the optimizer's pruning scheme. Framed against common PINN practice, this setup remains in the family of fully connected architectures typically paired with Adam optimization and standard initializations such as Glorot/Xavier and tanh activations, even as the specific activation choice can shift the minimum achievable error depending on the learning objective and the derivatives required at deployment time^{67–69}. In this context, the relatively small global sensitivity assigned to activation choice (4.8%) should not be read as indicating irrelevance, because other inverse/PINN formulations can exhibit robustness across activation choices depending on how the learning bias is imposed, but activation selection is still widely noted as important for extrapolation performance when the learned representation is used outside the training distribution⁷⁰.

Inverse dynamics via automatic differentiation

A practical contribution of the inverse-dynamics results is the strong evidence for a data-efficiency ‘knee’ at 4,000 samples under the study’s training and evaluation protocol (datasets of 1,000; 2,000; 4,000; 8,000; 16,000; 32,000; 64,000; 128,000; and 256,000 kinematic vectors), where increasing data volume from 1,000 to 4,000 produced a multi-fold improvement in median torque RMSE, but scaling further produced diminishing returns in accuracy relative to the added training burden. Although the 256,000-sample model achieved the lowest median errors (0.0118 Nm joint 2; 0.0189 Nm joint 1), the marginal gains were outweighed by the exponential increase in required data volume and training time, which rose from 29.6 seconds (1,000 samples) to 2.2 hours (256,000 samples). Importantly for control deployment, inference-time cost remained low and stable across training set sizes because the architecture was fixed, with a median inference latency of approximately 0.073 ms (mean 0.074 ± 0.052 ms), supporting the practical feasibility of embedding the inverse model in real-time control loops where inverse dynamics is routinely treated as an essential tool^{71,72}.

These empirical results also motivate selective comparisons to broader inverse-dynamics learning strategies. Pure black-box inverse dynamics networks can be difficult to train when physical-law compliance is required and training data provide incomplete coverage if the domain of interest. This has motivated physics-informed objectives for inverse identification rather than unconstrained regression⁷³. A complementary structure-preserving alternative is to enforce mechanical plausibility through constrained parameterizations, such as learning a lower-triangular factor to construct a positive definite mass–inertia matrix $M(q)$, and training via inverse dynamics to avoid numerical instabilities and computational costs associated with explicit matrix inversion⁷⁴. Finally, energy-based approaches show how differentiating learned scalar quantities can yield physically meaningful vector quantities (for example, potential forces computed as partial derivatives of a learned potential), supporting the broader design principle that learned models can preserve the mathematical structure commonly assumed in robots and mechanical-systems control rather than being purely black-box mappings^{75,76}.

Forward dynamics via optimization

The forward-dynamics evaluation provides an independent test of the predictive utility of our approach by comparing closed-loop simulation to analytical ground truth over a 5 s horizon across 50 independent trials. On accuracy, the forward solver achieved median angular RMSE of 0.0074 rad (joint 1) and 0.0119 rad (joint 2), with median velocity RMSEs of 0.0309 rad/s and 0.0677 rad/s and median acceleration RMSEs of 0.1653 rad/s² and 0.3821 rad/s² for joint 1 and joint 2, respectively. At the same operating point, the solver remained within the 2.0-ms real-time update budget, with a mean update time of 1.62 ms and median solver latency of 1.54 ms. These outcomes place the approach within a broader landscape of structured forward-dynamics learning in which physical structure can be embedded directly in the model class, such as Port-Hamiltonian neural ODE architectures designed to identify a Hamiltonian and associated physical matrices from observational data⁷⁷.

Equation-free real-time dynamics simulation

The real-time performance of the learned solver is best interpreted in relation to conventional physics engines, which remain the standard for fast forward simulation when a complete mechanical model is available. Modern engines such as MuJoCo are highly optimized for speed and accuracy, which makes them effective for online and large-scale simulation tasks that require many efficient rollouts, including distributed reinforcement learning, robotic skill acquisition and parallel trajectory optimization, where many rollouts must be executed efficiently^{78–80}. This performance, however, depends on a specified rigid-body model, where users provide physical parameters through a model description, after which the engine compiles the system and computes dynamics using optimized analytical algorithms. In contrast, our approach learns a scalar Appell-energy representation from kinematic-kinetic data and uses that representation to recover accelerations through optimization. It is therefore best viewed as a complementary route for cases in which the governing dynamics are difficult to write down completely, but paired motion and force or torque measurements are available.

613 **Implications for motor control hypotheses**

614 Internal-model theories in motor control distinguish forward and inverse internal models, where the forward
615 model predicts sensory outcomes by estimating causal relationships between system inputs and outputs,
616 and the inverse model provides motor commands that produce desired changes in state. Both types of
617 internal model depend on the dynamics of the motor system and must adapt as the motor apparatus⁸¹ or
618 environment⁸² changes. In this framing, the present results are naturally interpreted as a computational
619 analogue: the inverse-dynamics component yields accurate torque predictions at millisecond-scale latency
620 (median 0.073 ms), while the forward component supports multi-second state prediction with low angular
621 error across 50 independent 5-s trials. This dual use aligns with the general model-based control view that
622 either forward or inverse models are deployed depending on context and task requirements³¹, and it also
623 resonates with the motor-control perspective that predictive computation solves an equation of motion
624 forward in time, enabling internal forward modeling that can complement delayed sensory feedback⁸³.

625 At the same time, claims about neurobiological implementation should remain evidence-bound, because it
626 is controversial whether the cerebro-cerebellum provides inverse models or forward models for voluntary
627 limb movements and there is no experiment that unequivocally proves the cerebellum is the site of internal
628 models. Indeed, existing physiological and morphological evidence has been interpreted as supporting a
629 cerebellar forward model for limb movement, and Purkinje cell firing has been argued to exhibit
630 characteristics of a forward internal model while lacking dynamics-related signals required to be the output
631 of an inverse dynamics model^{84,85}. Computationally, the literature also emphasizes that coupling forward
632 and inverse models can facilitate learning³¹, because inverse models can learn context-appropriate control
633 commands when their paired forward model produces accurate predictions, which provides a disciplined
634 way to discuss why a shared, dynamics-consistent representation that supports both roles could be
635 advantageous without asserting direct equivalence to neural circuits.

Implications for system identification in robotics

A possible extension of the present work is to ask whether the network-training strategy used here could be used for system identification in robotics. Classical methods specify the rigid-body equations and estimate inertial, Coriolis, gravitational and related terms from motion–torque data^{86–88}, whereas data-driven methods often learn inverse dynamics directly from examples⁸⁹. Here, the learned object would instead be an Appell acceleration-energy scalar: measured torques would supervise its acceleration-gradient, rather than a direct torque map. This formulation could be useful when torques and kinematics are measurable, but the governing equations are difficult to specify, as in wearable robots. For example, exoskeleton data collected under controlled perturbations or structured assistance could be used to learn such a scalar without prescribing all inertial, interaction, frictional and compliant effects. The appeal is not that the present double-pendulum study establishes exoskeleton control, but that exoskeletons pose precisely the modeling challenges—sensor noise and delay, uncertain actuator torques, human–device interaction forces, transmission compliance, alignment variability and user-dependent dynamics—that can make unconstrained input–output networks brittle. Testing this extension would require calibrated torque measurements, explicit noise and filtering models, additional generalized forces, treatment of friction and compliance, scaling to higher-dimensional articulated systems and validation on closed-loop experimental trajectories.

Limitations

The most immediate limitation in the present results is that validation relied on analytically specified reference dynamics and numerical integration, which provides a controlled benchmark but does not yet test robustness to the measurement noise, calibration uncertainty, and unmodeled effects that arise with physical sensors and actuators. Specifically, test trajectories were generated through numerical integration of the equations of motion, and the reference integration used a fourth-order Runge–Kutta scheme with fixed time step of 0.1 ms, against which both inverse predictions and closed-loop forward rollouts were evaluated. Similarly, the empirical claims about data efficiency and training cost are bounded to the dataset

sizes explicitly studied (1,000; 2,000; 4,000; 8,000; 16,000; 32,000; 64,000; 128,000; and 256,000 kinematic vectors) and their associated compute trade-offs, CPU-measured increase in training time from 29.6 seconds (1,000 kinematic vectors) to 2.2 hours (256,000 kinematic vectors), so the extent to which these scaling relationships transfer to other training regimes remains an open question for future work.

Another limitation is that the present study was conducted on a planar two-link double-pendulum system. This model class has a productive history as a minimal test bed for multi-body dynamics; planar shoulder–elbow reaching studies demonstrated that intersegmental interaction torques require predictive compensation^{90,91}, and double-pendulum abstractions of the swing leg have been used to analyze the passive mechanical contributions to locomotion^{92–94}. The two-link system is therefore an established abstraction for isolating the nonlinear, coupled dynamics that make multi-body inverse and forward modeling non-trivial, and it provides a meaningful first test of whether the learned Gibbs-Appell representation can support both computation types. The present results establish feasibility within this low-dimensional setting. Extending the approach to higher-dimensional articulated bodies with joint constraints, compliance, contact, friction, actuator dynamics, and sensorimotor delays—and ultimately to systems governed by experimentally measured rather than analytically generated dynamics—remains an important open problem and a necessary step toward determining the practical scope of the approach.

Lastly, our results should not be interpreted as evidence that unconstrained neural networks cannot approximate the dynamics of a double pendulum. In this low-dimensional, noiseless setting, a sufficiently tuned multilayer perceptron (MLP) trained directly on $(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) \rightarrow \boldsymbol{\tau}$ may achieve low in-distribution torque error. The distinction we emphasize is instead representational. ANNet constrains the learned torque field to arise as the acceleration-gradient of a scalar Appell-energy function, allowing the same learned object to be used for inverse dynamics through automatic differentiation and for forward dynamics through optimization. This framing is consistent with structure-preserving neural dynamics models, including Hamiltonian Neural Networks, Lagrangian Neural Networks, and Deep Lagrangian Networks, where physical priors are introduced to bias learning toward conserved, mechanically plausible, or better-constrained dynamics rather than merely to improve pointwise regression accuracy^{75,76,95}. The derivative-

supervised loss used here is also related to Sobolev training, in which derivative information is incorporated into the learning objective to constrain the learned function beyond its scalar output values⁵⁴. Complementary work from our group further supports this representation-level distinction by showing that separating equation-of-motion approximation from numerical integration can improve forward-dynamics stability relative to a direct-mapping recurrent baseline under unseen perturbations⁹⁶.

Conclusions

This study shows that one neural network can be used to simulate both inverse and forward dynamics in a double-pendulum system. The network learned a single scalar quantity from classical mechanics, and its derivative provided accurate torque predictions. The same learned quantity also allowed the system to predict motion forward in time without training a separate forward model. These results suggest a practical way to link prediction and control in one physics-guided model, while future work will need to test the approach in larger systems and with experimental data.

Data Availability

The data supporting the findings of this study are available in the public GitHub repository <https://github.com/SerhiiBahdas/ms-appellian-neural-networks>. The repository contains the data files used to generate the main and supplementary figures. The datasets analyzed in this study were generated using the double-pendulum model described in the Methods.

Code Availability

The source code used to generate the figures is available in the public GitHub repository <https://github.com/SerhiiBahdas/ms-appellian-neural-networks>. Additional custom code used for neural-network training, hyperparameter optimization, and optimization-based forward-dynamics simulations is available from the corresponding author upon reasonable request.

Acknowledgement

This work was supported by internal funds from the Department of Mechanical Engineering and the Neuroscience Institute at Carnegie Mellon University. The content is solely the responsibility of the authors and does not necessarily represent the official views of the funders.

References

1. Grillner, S. *et al.* Neural networks that co-ordinate locomotion and body orientation in lamprey. *Trends Neurosci.* **18**, 270–279 (1995).
2. Grillner, S. & Wallén, P. How Does The Lamprey Central Nervous System Make The Lamprey Swim? *J. Exp. Biol.* **112**, 337–357 (1984).
3. Grillner, S. & El Manira, A. Current Principles of Motor Control, with Special Reference to Vertebrate Locomotion. *Physiol. Rev.* **100**, 271–320 (2020).
4. Sobinov, A. R. & Bensmaia, S. J. The neural mechanisms of manual dexterity. *Nat. Rev. Neurosci.* **22**, 741–757 (2021).
5. Ma, R. R. & Dollar, A. M. On dexterity and dexterous manipulation. in *2011 15th International Conference on Advanced Robotics (ICAR)* 1–7 (IEEE, Tallinn, Estonia, 2011). doi:10.1109/ICAR.2011.6088576.
6. Franklin, D. W. & Wolpert, D. M. Computational Mechanisms of Sensorimotor Control. *Neuron* **72**, 425–442 (2011).
7. Wolpert, D. M. & Ghahramani, Z. Computational principles of movement neuroscience. *Nat. Neurosci.* **3**, 1212–1217 (2000).
8. Theodorou, E. & Valero-Cuevas, F. J. Optimality in neuromuscular systems. in *2010 Annual International Conference of the IEEE Engineering in Medicine and Biology* 4510–4516 (2010). doi:10.1109/IEMBS.2010.5626055.

9. Neilson, P. D. & Neilson, M. D. Neural mechanisms for control of multivariable, redundant, nonlinear musculoskeletal systems. in *Proceedings of the 40th IEEE Conference on Decision and Control (Cat. No.01CH37228)* vol. 1 13–14 vol.1 (2001).
10. Prilutsky, B. I. & Zatsiorsky, V. M. Neural Control Principles: Bernstein's Insights from Biomechanics of Human Movement. in *Bernstein's Construction of Movements* (Routledge, 2020).
11. Ting, L. H. & Chiel, H. J. Muscle, Biomechanics, and Implications for Neural Control. in *Neurobiology of Motor Control* 365–416 (John Wiley & Sons, Ltd, 2017). doi:10.1002/9781118873397.ch12.
12. Latash, M. L. The bliss (not the problem) of motor abundance (not redundancy). *Exp. Brain Res.* **217**, 1–5 (2012).
13. Bernstein, N. *The Co-Ordination and Regulation of Movements*. (Pergamon Press, 1967).
14. Desmurget, M. & Grafton, S. Feedback or feedforward control: End of a dichotomy. in *Taking action: Cognitive neuroscience perspectives on intentional acts* 289–338 (The MIT Press, Cambridge, MA, US, 2003).
15. Schwartz, A. B. Movement: How the Brain Communicates with the World. *Cell* **164**, 1122–1135 (2016).
16. Hinder, M. R., Tresilian, J. R., Riek, S. & Carson, R. G. The contribution of visual feedback to visuomotor adaptation: How much and when? *Brain Res.* **1197**, 123–134 (2008).
17. Saunders, J. A. & Knill, D. C. Humans use continuous visual feedback from the hand to control fast reaching movements. *Exp. Brain Res.* **152**, 341–352 (2003).
18. Saunders, J. A. & Knill, D. C. Visual Feedback Control of Hand Movements. *J. Neurosci.* **24**, 3223–3234 (2004).
19. Jeka, J. J. & Lackner, J. R. Fingertip contact influences human postural control. *Exp. Brain Res.* **100**, 495–502 (1994).
20. Prochazka, A. Proprioceptive Feedback and Movement Regulation. *Compr. Physiol.* **1995**, 89–127 (1995).
21. Feldman, A. G. Once more on the equilibrium-point hypothesis (lambda model) for motor control. *J. Mot. Behav.* **18**, 17–54 (1986).

- 758 22. Scott, S. H., Cluff, T., Lowrey, C. R. & Takei, T. Feedback control during voluntary motor actions. *Curr.*
759 *Opin. Neurobiol.* **33**, 85–94 (2015).
- 760 23. Nijhawan, R. & Wu, S. Compensating time delays with neural predictions: are predictions sensory or
761 motor? *Philos. Trans. R. Soc. Math. Phys. Eng. Sci.* **367**, 1063–1078 (2009).
- 762 24. Desmurget, M. & Grafton, S. Forward modeling allows feedback control for fast reaching movements.
763 *Trends Cogn. Sci.* **4**, 423–431 (2000).
- 764 25. Seidler, R. D., Noll, D. C. & Thiers, G. Feedforward and feedback processes in motor control.
765 *NeuroImage* **22**, 1775–1783 (2004).
- 766 26. Shadmehr, R., Smith, M. A. & Krakauer, J. W. Error Correction, Sensory Prediction, and Adaptation in
767 Motor Control. *Annu. Rev. Neurosci.* **33**, 89–108 (2010).
- 768 27. Diedrichsen, J., Shadmehr, R. & Ivry, R. B. The coordination of movement: optimal feedback control
769 and beyond. *Trends Cogn. Sci.* **14**, 31–39 (2010).
- 770 28. Todorov, E. & Jordan, M. I. Optimal feedback control as a theory of motor coordination. *Nat. Neurosci.*
771 **5**, 1226–1235 (2002).
- 772 29. Kawato, M. & Wolpert, D. Internal Models for Motor Control. in *Novartis Foundation Symposium 218 -*
773 *Sensory Guidance of Movement* 291–307 (John Wiley & Sons, Ltd).
774 doi:10.1002/9780470515563.ch16.
- 775 30. Kawato, M. Internal models for motor control and trajectory planning. *Curr. Opin. Neurobiol.* **9**, 718–
776 727 (1999).
- 777 31. Wolpert, D. M. & Kawato, M. Multiple paired forward and inverse models for motor control. *Neural Netw.*
778 **11**, 1317–1329 (1998).
- 779 32. Blakemore, S. J., Wolpert, D. M. & Frith, C. D. Central cancellation of self-produced tickle sensation.
780 *Nat. Neurosci.* **1**, 635–640 (1998).
- 781 33. Shadmehr, R., Brashers-Krug, T. & Mussa-Ivaldi, F. Interference in Learning Internal Models of Inverse
782 Dynamics in Humans. in *Advances in Neural Information Processing Systems* vol. 7 (MIT Press, 1994).
- 783 34. Shidara, M., Kawano, K., Gomi, H. & Kawato, M. Inverse-dynamics model eye movement control by
784 Purkinje cells in the cerebellum. *Nature* **365**, 50–52 (1993).

35. Schweighofer, N., Arbib, M. A. & Kawato, M. Role of the cerebellum in reaching movements in humans. I. Distributed inverse dynamics control. *Eur. J. Neurosci.* **10**, 86–94 (1998).
36. Lisberger, S. G. Internal models of eye movement in the floccular complex of the monkey cerebellum. *Neuroscience* **162**, 763–776 (2009).
37. Kawato, M., Ohmae, S., Hoang, H. & Sanger, T. 50 Years Since the Marr, Ito, and Albus Models of the Cerebellum. *Neuroscience* **462**, 151–174 (2021).
38. Honda, T. *et al.* Tandem internal models execute motor learning in the cerebellum. *Proc. Natl. Acad. Sci.* **115**, 7428–7433 (2018).
39. Gentili, R. J. *et al.* Integration of Gravitational Torques in Cerebellar Pathways Allows for the Dynamic Inverse Computation of Vertical Pointing Movements of a Robot Arm. *PLOS ONE* **4**, e5176 (2009).
40. Laurens, J. & Angelaki, D. E. A unified internal model theory to resolve the paradox of active versus passive self-motion sensation. *eLife* **6**, e28074 (2017).
41. Featherstone, R. *Rigid Body Dynamics Algorithms*. (Springer US, Boston, MA, 2008). doi:10.1007/978-1-4899-7560-7.
42. Smith, M. A. & Shadmehr, R. Intact ability to learn internal models of arm dynamics in Huntington's disease but not cerebellar degeneration. *J. Neurophysiol.* **93**, 2809–2821 (2005).
43. Izawa, J., Criscimagna-Hemminger, S. E. & Shadmehr, R. Cerebellar contributions to reach adaptation and learning sensory consequences of action. *J. Neurosci. Off. J. Soc. Neurosci.* **32**, 4230–4239 (2012).
44. Ghasia, F. F., Meng, H. & Angelaki, D. E. Neural Correlates of Forward and Inverse Models for Eye Movements: Evidence from Three-Dimensional Kinematics. *J. Neurosci.* **28**, 5082–5087 (2008).
45. de Graaf, M. L. *et al.* InFoRM: a unified inverse and forward model for sensorimotor control. *Sci. Rep.* **16**, 8490 (2026).
46. Todorov, E. & Ghahramani, Z. Unsupervised learning of sensory-motor primitives. in *Proceedings of the 25th Annual International Conference of the IEEE Engineering in Medicine and Biology Society* vol. 2 1750-1753 Vol.2 (2003).

811 47. Diedrichsen, J., Hashambhoy, Y., Rane, T. & Shadmehr, R. Neural correlates of reach errors. *J.*
812 *Neurosci. Off. J. Soc. Neurosci.* **25**, 9919–9931 (2005).

813 48. Gauss, C. F. Über ein neues allgemeines Grundgesetz der Mechanik. **1829**, 232–235 (1829).

814 49. Appell, P. *Sur une forme générale des équations de la dynamique*. (Gauthier-Villars et cie, Paris, 1925).

815 50. Pars, L. A. *A Treatise on Analytical Dynamics*. (London, Heinemann, 1965).

816 51. Gibbs, J. W. *On the Fundamental Formulae of Dynamics*. (American Journal of Mathematics, 1879).

817 52. Papastavridis, J. G. *Analytical Mechanics: A Comprehensive Treatise on the Dynamics of Constrained*
818 *Systems*. (World scientific, Hackensack, 2014).

819 53. Hendrycks, D. & Gimpel, K. Gaussian Error Linear Units (GELUs). Preprint at
820 <https://doi.org/10.48550/arXiv.1606.08415> (2023).

821 54. Czarnecki, W. M., Osindero, S., Jaderberg, M., Swirszcz, G. & Pascanu, R. Sobolev Training for Neural
822 Networks. in *Advances in Neural Information Processing Systems* vol. 30 (Curran Associates, Inc.,
823 2017).

824 55. Akiba, T., Sano, S., Yanase, T., Ohta, T. & Koyama, M. Optuna: A Next-generation Hyperparameter
825 Optimization Framework. Preprint at <https://doi.org/10.48550/arXiv.1907.10902> (2019).

826 56. Kingma, D. P. & Ba, J. Adam: A Method for Stochastic Optimization. Preprint at
827 <https://doi.org/10.48550/arXiv.1412.6980> (2017).

828 57. Allen, S. J., Yeadon, M. R. (Fred) & King, M. A. The effect of increasing strength and approach velocity
829 on triple jump performance. *J. Biomech.* **49**, 3796–3802 (2016).

830 58. Owaki, D., Horikiri, S., Nishii, J. & Ishiguro, A. Tegotae-Based Control Produces Adaptive Inter- and
831 Intra-limb Coordination in Bipedal Walking. *Front. Neurobotics* **15**, (2021).

832 59. Kawato, M., Furukawa, K. & Suzuki, R. A hierarchical neural-network model for control and learning of
833 voluntary movement. *Biol. Cybern.* **57**, 169–185 (1987).

834 60. Virtanen, P. *et al.* SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nat. Methods*
835 **17**, 261–272 (2020).

836 61. Seabold, S. & Perktold, J. Statsmodels: Econometric and Statistical Modeling with Python. *SciPy 2010*
837 <https://doi.org/10.25080/Majora-92bf1922-011> (2010) doi:10.25080/Majora-92bf1922-011.

62. Paszke, A. *et al.* PyTorch: An Imperative Style, High-Performance Deep Learning Library. Preprint at <https://doi.org/10.48550/arXiv.1912.01703> (2019).
63. Otten, E. Inverse and forward dynamics: models of multi-body systems. *Philos. Trans. R. Soc. B Biol. Sci.* **358**, 1493–1500 (2003).
64. Krishnapriyan, A. S., Gholami, A., Zhe, S., Kirby, R. M. & Mahoney, M. W. Characterizing possible failure modes in physics-informed neural networks. in (2021).
65. Wang, S., Li, B., Chen, Y. & Perdikaris, P. PirateNets: Physics-informed Deep Learning with Residual Adaptive Networks. Preprint at <https://doi.org/10.48550/arXiv.2402.00326> (2024).
66. Pantidis, P., Eldababy, H., Tagle, C. M. & Mobasher, M. E. Error convergence and engineering-guided hyperparameter search of PINNs: Towards optimized I-FENN performance. *Comput. Methods Appl. Mech. Eng.* **414**, 116160 (2023).
67. Dashtbayaz, N. H., Farhani, G., Wang, B. & Ling, C. X. Physics-Informed Neural Networks: Minimizing Residual Loss with Wide Networks and Effective Activations. Preprint at <https://doi.org/10.48550/arXiv.2405.01680> (2024).
68. Wang, H., Lu, L., Song, S. & Huang, G. Learning Specialized Activation Functions for Physics-informed Neural Networks. *Commun. Comput. Phys.* **34**, 869–906 (2023).
69. Farea, A. & Celebi, M. S. Learnable activation functions in physics-informed neural networks for solving partial differential equations. *Comput. Phys. Commun.* **315**, 109753 (2025).
70. Jagtap, A. D. & Karniadakis, G. E. Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. *J. Comput. Phys.* **404**, 109136 (2020).
71. Polydoros, A., Boukas, E. & Nalpantidis, L. Online multi-target learning of inverse dynamics models for computed-torque control of compliant manipulators. in 4716–4722 (2017). doi:10.1109/IROS.2017.8206344.
72. Zhang, Y. *et al.* Invertible liquid neural network-based learning of inverse kinematics and dynamics for robotic manipulators. *Sci. Rep.* **15**, 42311 (2025).

73. Bolderman, M., Fan, D., Lazar, M. & Butler, H. Generalized feedforward control using physics-informed neural networks: 18th IFAC Workshop on Control Applications of Optimization, CAO 2022. *IFAC-Pap.* **55**, 148–153 (2022).
74. Lutter, M. & Peters, J. Combining physics and deep learning to learn continuous-time dynamics models. *Int. J. Robot. Res.* **42**, 83–107 (2023).
75. Greydanus, S., Dzamba, M. & Yosinski, J. Hamiltonian Neural Networks. Preprint at <https://doi.org/10.48550/arXiv.1906.01563> (2019).
76. Cranmer, M. *et al.* Lagrangian Neural Networks. Preprint at <https://doi.org/10.48550/arXiv.2003.04630> (2020).
77. Duong, T., Altawaitan, A., Stanley, J. & Atanasov, N. Port-Hamiltonian Neural ODE Networks on Lie Groups For Robot Dynamics Learning and Control. Preprint at <https://doi.org/10.48550/arXiv.2401.09520> (2024).
78. Todorov, E., Erez, T. & Tassa, Y. MuJoCo: A physics engine for model-based control. in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems* 5026–5033 (2012). doi:10.1109/IROS.2012.6386109.
79. Makoviychuk, V. *et al.* Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning. *ArXiv* <https://www.semanticscholar.org/paper/Isaac-Gym%3A-High-Performance-GPU-Based-Physics-For-Makoviychuk-Wawrzyniak/49142e3e381c0dc7fee0049ea41d2ef02c0340d7> (2021).
80. Rudin, N., Hoeller, D., Reist, P. & Hutter, M. Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning. *ArXiv* <https://www.semanticscholar.org/paper/Learning-to-Walk-in-Minutes-Using-Massively-Deep-Rudin-Hoeller/ca6096142016a2ba8133f6cb2c04ad30f5eae730> (2021).
81. Herrick, E. M. & Yakovenko, S. Evidence of sensory error threshold in triggering locomotor adaptations in humans. *PLOS ONE* **20**, e0321949 (2025).
82. Shadmehr, R. & Mussa-Ivaldi, F. A. Adaptive representation of dynamics during learning of a motor task. *J. Neurosci.* **14**, 3208–3224 (1994).

889 83. Wolpert, D. M. & Miall, R. C. Forward Models for Physiological Motor Control. *Neural Netw. Off. J. Int.*
890 *Neural Netw. Soc.* **9**, 1265–1279 (1996).

891 84. Ishikawa, T., Tomatsu, S., Izawa, J. & Kakei, S. The cerebro-cerebellum: Could it be loci of forward
892 models? *Neurosci. Res.* **104**, 72–79 (2016).

893 85. Ebner, T. J. & Pasalar, S. Cerebellum Predicts the Future Motor State. *The Cerebellum* **7**, 583–588
894 (2008).

895 86. Atkeson, C. G., An, C. H. & Hollerbach, J. M. Estimation of Inertial Parameters of Manipulator Loads
896 and Links. *Int. J. Robot. Res.* **5**, 101–119 (1986).

897 87. Gautier, M. & Khalil, W. Direct calculation of minimum set of inertial parameters of serial robots. *IEEE*
898 *Trans. Robot. Autom.* **6**, 368–373 (1990).

899 88. Wu, J., Wang, J. & You, Z. An overview of dynamic parameter identification of robots. *Robot. Comput.-*
900 *Integr. Manuf.* **26**, 414–419 (2010).

901 89. Manukian, M., Bahdasariants, S. & Yakovenko, S. Artificial physics engine for real-time inverse
902 dynamics of arm and hand movement. *PLOS ONE* **18**, e0295750 (2023).

903 90. Hollerbach, J. M. & Flash, T. Dynamic interactions between limb segments during planar arm
904 movement. *Biol. Cybern.* **44**, 67–77 (1982).

905 91. Gribble, P. L. & Ostry, D. J. Compensation for Interaction Torques During Single- and Multijoint Limb
906 Movement. *J. Neurophysiol.* **82**, 2310–2326 (1999).

907 92. Mochon, S. & McMahon, T. A. Ballistic walking: an improved model. *Math. Biosci.* **52**, 241–260 (1980).

908 93. McGeer, T. Passive walking with knees. in , *IEEE International Conference on Robotics and*
909 *Automation Proceedings* 1640–1645 vol.3 (1990). doi:10.1109/ROBOT.1990.126245.

910 94. Garcia, M., Chatterjee, A., Ruina, A. & Coleman, M. The Simplest Walking Model: Stability, Complexity,
911 and Scaling. *J. Biomech. Eng.* **120**, 281–288 (1998).

912 95. Lutter, M., Ritter, C. & Peters, J. Deep Lagrangian Networks: Using Physics as Model Prior for Deep
913 Learning. Preprint at <https://doi.org/10.48550/arXiv.1907.04490> (2019).

914 96. Bahdasariants, S. & Yakovenko, S. Neural networks learn forward dynamics when freed from numerical
915 integration. Preprint at (2026).

Supplementary information

Forward dynamics equations

Let $\mathbf{q} = [q_1, q_2]^T$ denote joint angles and $\dot{\mathbf{q}} = [\dot{q}_1, \dot{q}_2]^T$ denote angular velocities. From the Gibbs-Appell formulation, the partial derivative of the Appell acceleration energy with respect to generalized acceleration can be written as:

$$\frac{\partial S}{\partial \ddot{\mathbf{q}}} = M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}),$$

where $M(\mathbf{q})$ is the configuration-dependent mass matrix and $C(\mathbf{q}, \dot{\mathbf{q}})$ contains velocity-dependent inertial terms. Setting this expression equal to the negative gradient of the potential energy gives:

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}) = -G(\mathbf{q}),$$

where $G(\mathbf{q}) = \partial V / \partial \mathbf{q}$. Thus, the ground-truth forward dynamics used for numerical integration were obtained as:

$$\ddot{\mathbf{q}} = -M(\mathbf{q})^{-1}[C(\mathbf{q}, \dot{\mathbf{q}}) + G(\mathbf{q})].$$

For the two-link system, the matrix and vectors were:

$$M(\mathbf{q}) = \begin{bmatrix} l_1^2(m_1 + m_2) + 2l_1l_2m_2 \cos q_2 + l_2^2m_2 & l_2m_2(l_1 \cos q_2 + l_2) \\ l_2m_2(l_1 \cos q_2 + l_2) & l_2^2m_2 \end{bmatrix},$$

$$C(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} -l_1l_2m_2 \sin q_2 (2\dot{q}_1\dot{q}_2 + \dot{q}_2^2) \\ l_1l_2m_2 \sin q_2 \dot{q}_1^2 \end{bmatrix},$$

$$G(\mathbf{q}) = \begin{bmatrix} g[l_1(m_1 + m_2) \sin q_1 + l_2m_2 \sin(q_1 + q_2)] \\ gl_2m_2 \sin(q_1 + q_2) \end{bmatrix}.$$

Equivalently, defining $D = m_{11}m_{22} - m_{12}^2 = l_1^2l_2^2m_2(m_1 + m_2 \sin^2 q_2)$, the joint accelerations were:

$$\ddot{q}_1 = \frac{-m_{22}(C_1 + G_1) + m_{12}(C_2 + G_2)}{D},$$

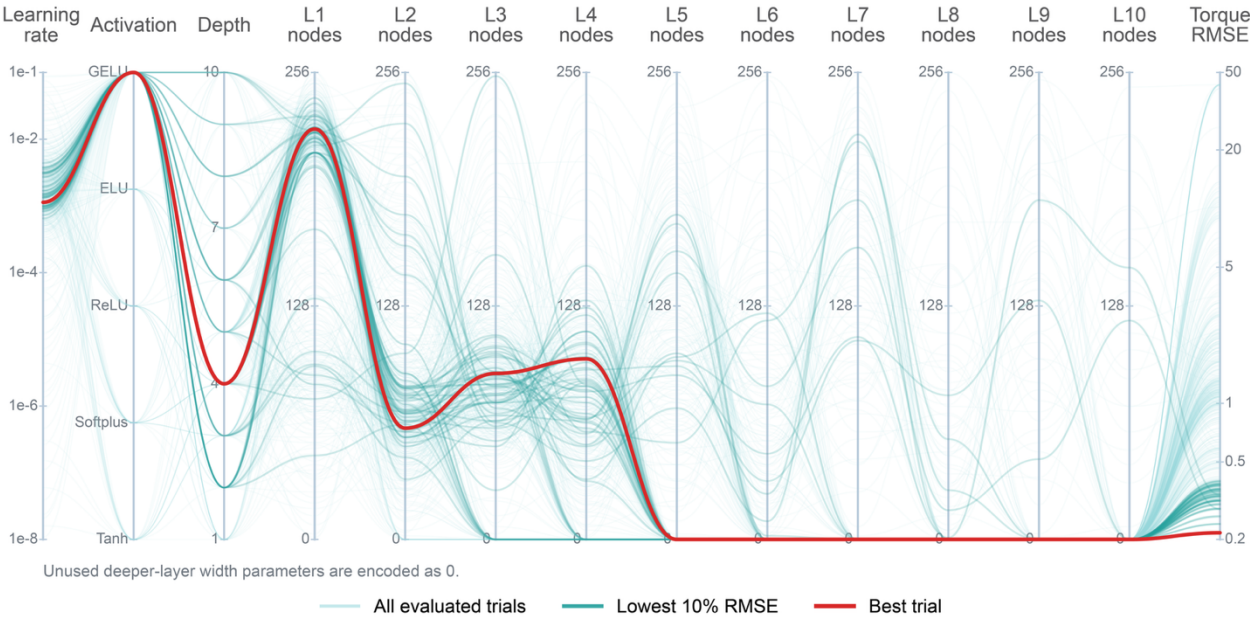
$$\ddot{q}_2 = \frac{m_{12}(C_1 + G_1) - m_{11}(C_2 + G_2)}{D}.$$

These accelerations were then reformulated as the first-order system:

$$\frac{d}{dt} \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \ddot{q}_1(\mathbf{q}, \dot{\mathbf{q}}) \\ \ddot{q}_2(\mathbf{q}, \dot{\mathbf{q}}) \end{bmatrix},$$

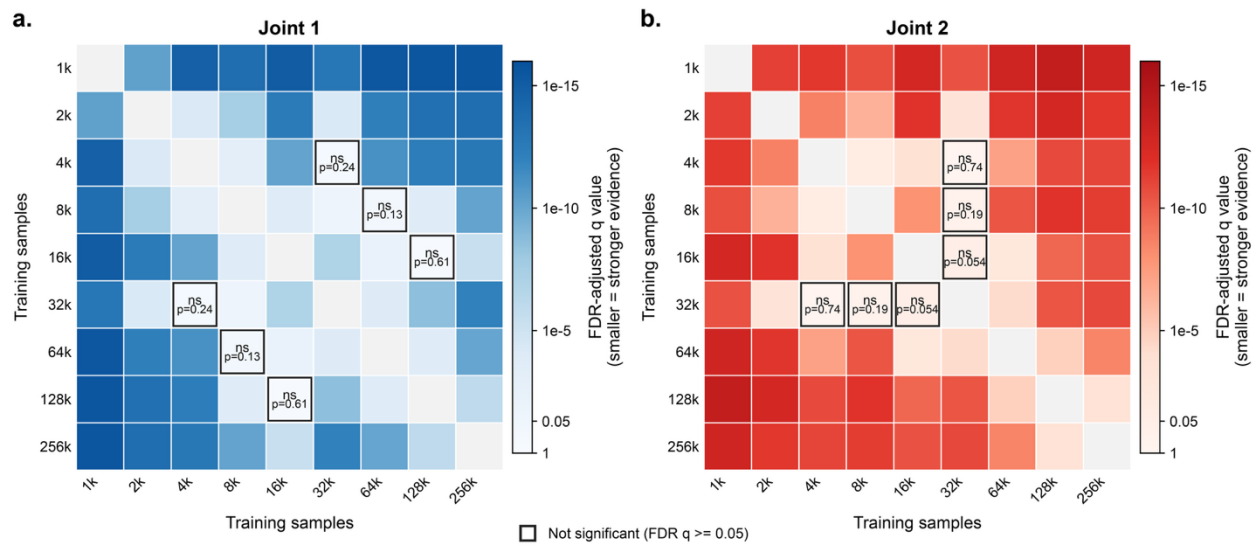
which was integrated using the fixed-step fourth-order Runge-Kutta scheme.

Supplementary Figures and Legends



Supplementary Fig. 1 | Parallel-coordinate representation of hyperparameter optimization.

Each polyline represents one Optuna hyperparameter trial evaluated during the architecture search. Axes show the learning rate, activation function, network depth, hidden-layer widths, and validation torque RMSE. Pale cyan lines show all evaluated trials, teal lines indicate the lowest 10% of trials by RMSE, and the red line marks the best-performing configuration used for subsequent analyses. Unused deeper-layer width parameters in shallower networks are encoded as 0 to display variable-depth architectures on a common axis set.



Supplementary Fig. 2 | Pairwise statistical comparison of inverse-dynamics accuracy across training-set sizes.

a,b, Matrix representation of pairwise comparisons of trajectory-wise torque RMSE distributions for joint 1 (a) and joint 2 (b) across training-set sizes. Rows and columns denote the dataset sizes being compared. Pairwise differences were evaluated using two-sided Mann-Whitney U tests with Benjamini-Hochberg false discovery rate correction. Cell color reports the FDR-adjusted p value, with darker blue for joint 1 and darker red for joint 2 indicating stronger evidence that the corresponding RMSE distributions differ. Gray diagonal cells represent self-comparisons. Outlined cells labeled ns denote comparisons that were not significant after FDR correction ($p \geq 0.05$).